Majority Element

AlgoExpert · Medium · Arrays

The problem

- An element occurs in more than half the array; find it (Boyer-Moore voting).

Intuition

- Boyer-Moore voting: pair off different values; the majority survives the cancellation.

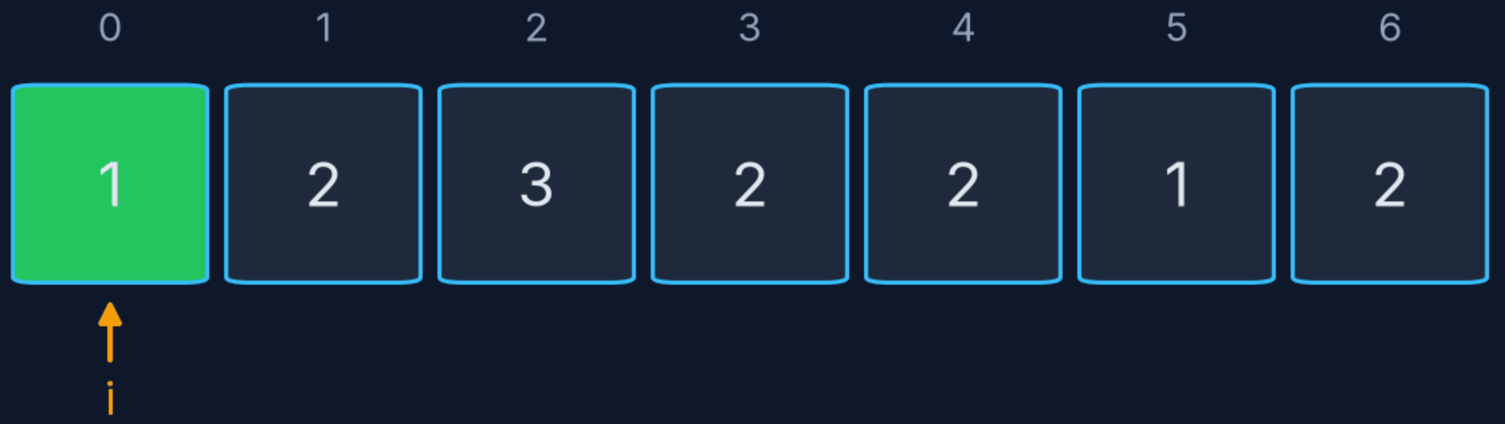
Brute force vs optimal

- Brute force: count each value → $O(n^2)$
- Hash map counts → O(n) time, O(n) space
- Sort and take the middle → O(n log n)
- Boyer–Moore voting → O(n) time, O(1) space

Approach

- 1. Keep a candidate and a count
- 2. Same value increments, different decrements
- 3. When the count hits zero, adopt the current value
- 4. The final candidate is the answer

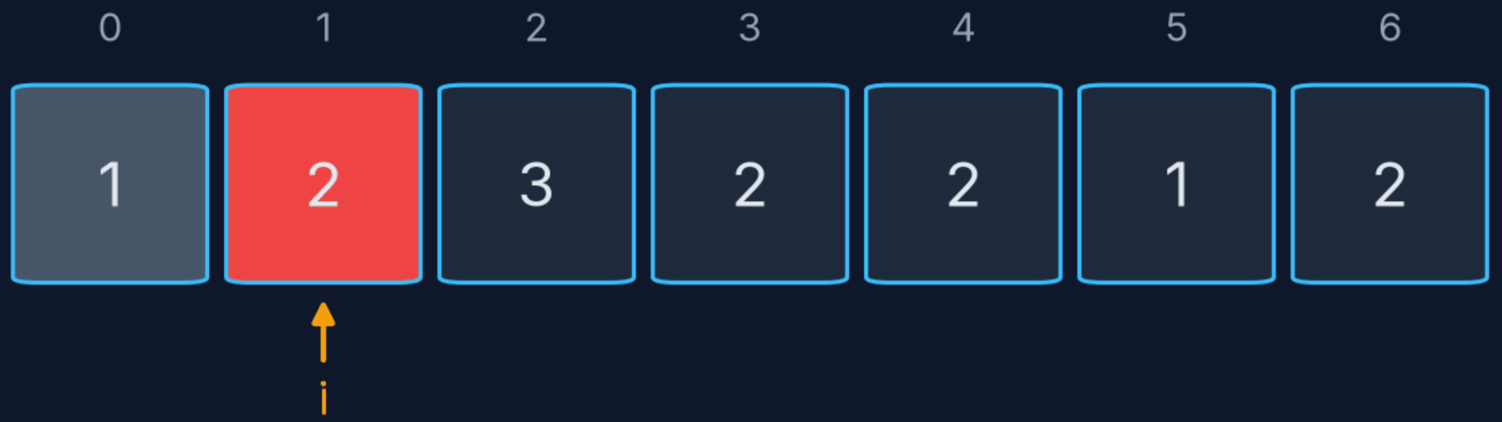
Worked example




```
candidate = 1
count = 1
```

1 matches → candidate 1, count 1

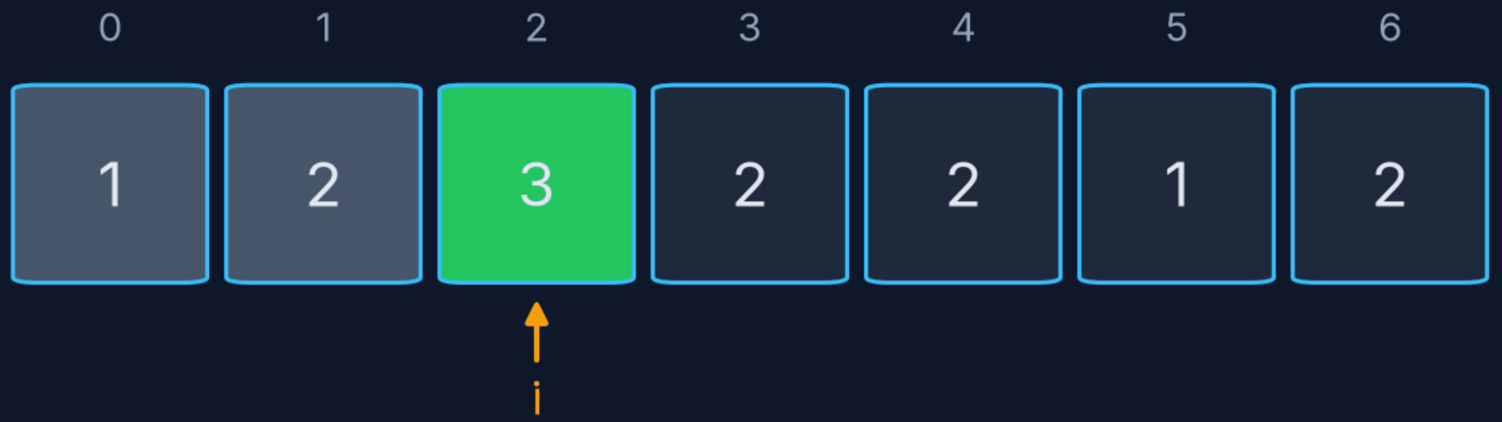
Worked example




```
candidate = 1
count = 0
```

2 cancels → candidate 1, count 0

Worked example




```
candidate = 3
count = 1
```

3 matches → candidate 3, count 1

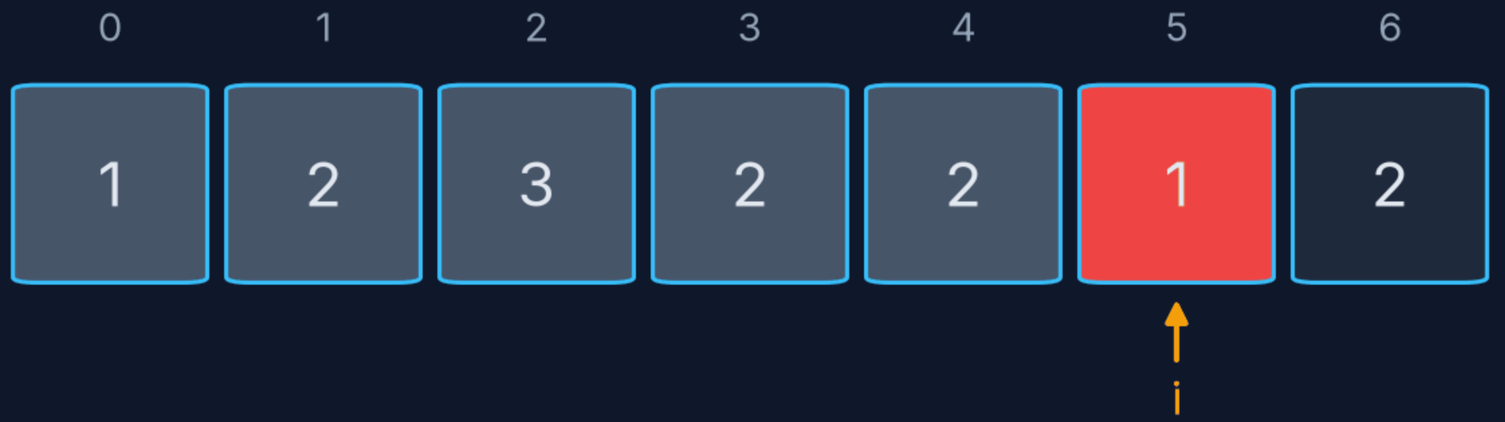
Worked example




```
candidate = 2
count = 1
```

2 matches → candidate 2, count 1

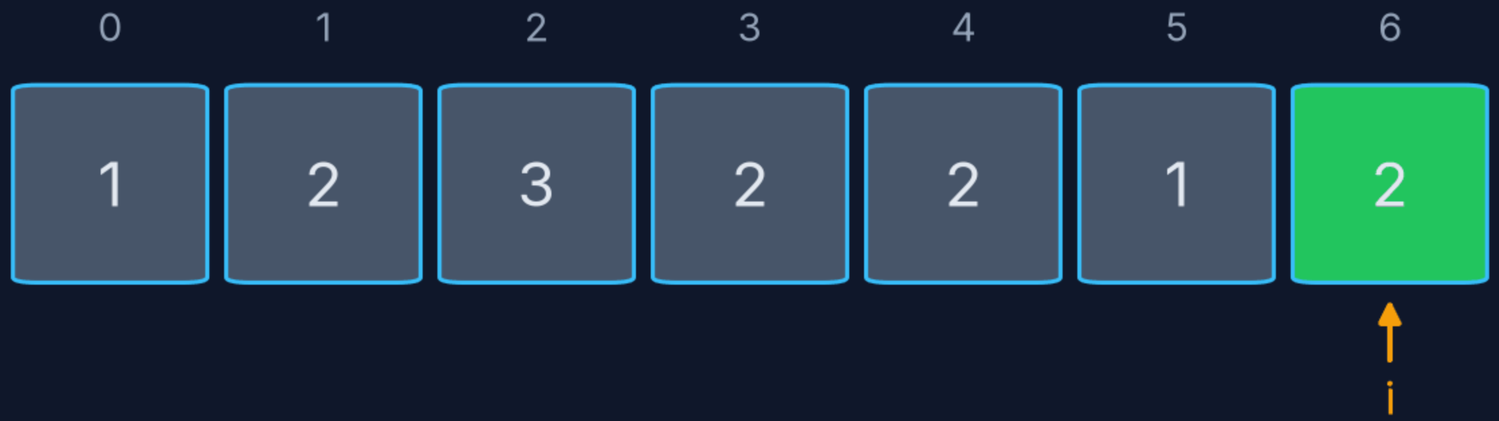
Worked example




```
candidate = 2
count = 0
```

1 cancels → candidate 2, count 0

Worked example




```
candidate = 2
count = 1
```

2 matches → candidate 2, count 1

Java solution

```java
public class MajorityElement {
    public static int majorityElement(int[] a) {
        int cand = 0, count = 0;
        for (int x : a) {
            if (count == 0) cand = x;
            count += (x == cand) ? 1 : -1;
        }
        return cand;
    }
}
```

Complexity

- Time: O(n)
- Space: O(1)

Edge cases & pitfalls

- Guaranteed majority exists here; otherwise verify with a 2nd pass
- Count resets don't lose the answer
- Single element

Interview tips & follow-ups

- Explain the cancellation intuition
- Follow-up: elements appearing > n/3 times (two candidates)

Recap

- Boyer-Moore voting: pair off different values; the majority survives the cancellation.
- Complexity: O(n)